

```
interface.py
1  # Import search API from a package called 'sw'
2
28  film_list = search_api.parse_films(char)
29  titles = search_api.format_titles(film_list)
30  description = search_api.person_description(char_name, planet, titles)
31  print(description)
32
33  def check_db(chars):
34      for char in chars:
35          char_name = search_api.parse_name(char)
36          results = db.search(User.name == char_name)
37          if not results:
38              description = parse_char(char)
39          else:
40              name = results[0]['name']
41              planet = results[0]['planet']
42              titles = results[0]['titles']
43              description = search_api.person_description(name, planet, titles)
44              print(description)
45
46  if __name__ == '__main__':
47      fire.Fire(search)
```

```

(base) codio@cablnetchef-igloofocus:~/workspace$ python sw_characters/interface.py
Traceback (most recent call last):
  File ~/home/codio/workspace/sw_characters/interface.py, line 1, in <module>
    import fire
ModuleNotFoundError: No module named 'fire'
(base) codio@cablnetchef-igloofocus:~/workspace$ conda activate sw
(sw) codio@cablnetchef-igloofocus:~/workspace$ python sw_characters/interface.py
Traceback (most recent call last):
  File ~/home/codio/workspace/sw_characters/interface.py, line 2, in <module>
    from tinydb import TinyDB, Query
ModuleNotFoundError: No module named 'tinydb'
(sw) codio@cablnetchef-igloofocus:~/workspace$ conda install --channel=conda-forge tinydb
Collecting package metadata (current_repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
  current version: 4.10.3
  latest version: 23.5.2

Please update conda by running

$ conda update -n base -c defaults conda
```

Improving Performance

Too Many API Calls

Activate the virtual environment.

```
conda activate sw
```

Accessing the network is a relatively slow process for a computer. At a minimum, every character will make three API calls. If they appear in more films, then the script makes more API calls.

If we want to reduce the number of API calls, then we need to store the information locally. This is the benefit of using a database. Before accessing the network, first query the database for a matching record. If it exists, read from the database. If not, get the information from the API and then add it to the database. The more searches are made, the more information is stored locally, the faster the script runs.

We are going to use TinyDB for this project. There are many databases that work well with Python. Many of the databases are more robust and faster. However, we are going to keep things simple. With TinyDB, all data is stored in a local JSON file. The database package provides you with a nice interface for accessing and modifying the information in the database.

TinyDB is found the in the `conda-forge` channel, so the installation will look a bit different.

```
conda install --channel=conda-forge tinydb
```

Implementing the Database

Once the database installed, we need to import `TinyDB` and `Query` from the `tinydb` package.

```
import fire
from tinydb import TinyDB, Query
import search_api
```

Next, instantiate our database with the `db.json` file. Python will create this file for us. Then we need to instantiate a `Query` object. This will allow us to search the database.

```
db = TinyDB('db.json')
User = Query()
```

Find the `search` function. If the API returns information, we want to check if it exists in the database. Replace the call to the `parse_char_list` function with a call to the `check_db` function.

```
def search(name='luke'):
    characters = search_api.search(name)
    if characters is not None:
        check_db(characters)
    else:
        print(f'Cannot find the character "{name}"')
```

The `check_db` function takes a list of Star Wars characters. Iterate over the list, extracting the name of the character. Use the `search` method on the database object to see if it storing the user name. The `search` method returns a list of database records. Determine if the list is empty. If yes, create the `description` variable by calling the `parse_char` function. This is the slow part of our script as all of the API calls will be made.

```
def check_db(chars):
    for char in chars:
        char_name = search_api.parse_name(char)
        results = db.search(User.name == char_name)
        if not results:
            description = parse_char(char)
        else:
            name = results[0]['name']
            planet = results[0]['planet']
            titles = results[0]['titles']
            description = search_api.person_description(name, planet, titles)
            print(description)
```

If the database search does not return an empty list, extract the values associated with the `name`, `planet`, and `titles` keys. Create the `description` variable by calling `person_description` function from the `search_api` module. In short, collect the information from the database, not from the API. Finally, print the description. The print statement should not be inside the `else` branch.

```
def check_db(chars):
    for char in chars:
        char_name = search_api.parse_name(char)
        results = db.search(User.name == char_name)
        if not results:
            description = parse_char(char)
        else:
            name = results[0]['name']
            planet = results[0]['planet']
            titles = results[0]['titles']
            description = search_api.person_description(name, planet, titles)
            print(description)
```

The `parse_char` function is only called if the character is not in the database. Before returning the description, add a line that inserts the character name, planet, and formatted titles to the database so we no longer have to make an API call for that character.

```
def parse_char(char):
    char_name = search_api.parse_name(char)
    planet = search_api.parse_planet(char)
    film_list = search_api.parse_films(char)
    titles = search_api.format_titles(film_list)
    description = search_api.person_description(char_name, planet, titles)
    db.insert({'name':char_name, 'planet':planet, 'titles':titles})
    return description
```

▼ Code

Your code should look like this:

```
import fire
from tinydb import TinyDB, Query
import search_api

db = TinyDB('db.json')
User = Query()

def search(name='luke'):
    characters = search_api.search(name)
    if characters is not None:
        check_db(characters)
    else:
        print(f'Cannot find the character "{name}"')
```

```
def parse_char(char):
    char_name = search_api.parse_name(char)
    planet = search_api.parse_planet(char)
    film_list = search_api.parse_films(char)
    titles = search_api.format_titles(film_list)
    description = search_api.person_description(char_name, planet, titles)
    db.insert({'name':char_name, 'planet':planet, 'titles':titles})
    return description

def parse_char_list(chars):
    for char in chars:
        char_name = search_api.parse_name(char)
        planet = search_api.parse_planet(char)
        film_list = search_api.parse_films(char)
        titles = search_api.format_titles(film_list)
        description = search_api.person_description(char_name, planet, titles)
        print(description)

def check_db(chars):
    for char in chars:
        char_name = search_api.parse_name(char)
        results = db.search(User.name == char_name)
        if not results:
            description = parse_char(char)
        else:
            name = results[0]['name']
            planet = results[0]['planet']
            titles = results[0]['titles']
            description = search_api.person_description(name, planet, titles)
            print(description)

if __name__ == '__main__':
    fire.Fire(search)
```

Deactivate the virtual environment.

```
conda deactivate
```

Lab Question

Select the command that would use Poetry to install the `tinydb` package.

☐ poetry new tinydb

☐ poetry install tinydb

☒ poetry add tinydb

☐ poetry --install=tinydb

The correct answer is:

poetry add tinydb

Poetry uses the `add` command to install a new package. Because Poetry uses PyPi you do not need to specify a channel when installing `tinydb`.

Check It